

CBP: Coordinated management of cache partitioning, bandwidth partitioning and prefetch throttling

Nadja Ramhöj Holtryd, Madhavan Manivannan, Per Stenström, Miquel Pericàs

Department of Computer Science and Engineering

Chalmers University of Technology

Göteborg, Sweden

Email: {holtryd, madhavan, per.stenstrom, miquelp}@chalmers.se

Abstract—Reducing the average memory access time is crucial for improving the performance of applications running on multi-core architectures. With workload consolidation this becomes increasingly challenging due to shared resource contention. Techniques for partitioning of shared resources - cache and bandwidth - and prefetch throttling have been proposed to mitigate contention and reduce the average memory access time. However, existing proposals only employ a single or a subset of these techniques and are therefore not able to exploit the full potential of coordinated management of cache, bandwidth and prefetching. Our characterization results show that application performance, in several cases, is sensitive to prefetching, cache and bandwidth allocation altogether. Furthermore, the results show that managing these together provides higher performance potential during workload consolidation as it enables more resource trade-offs. In this paper, we propose CBP a coordination mechanism for dynamically managing prefetching throttling, cache and bandwidth partitioning, in order to reduce average memory access time and improve performance. CBP works by employing individual resource managers to determine the appropriate setting for each resource and a coordinating mechanism in order to enable inter-resource trade-offs. Our evaluation on a 16-core CMP shows that CBP, on average, improves performance by 11% compared to the state-of-the-art technique that manages cache partitioning and prefetching and by 50% compared to the baseline without cache partitioning, bandwidth partitioning and prefetch throttling.

I. INTRODUCTION

Memory access time has a significant impact on application performance. Effective utilization of the memory system is therefore necessary. Typically, resources in the memory system (e.g., last-level cache (LLC) and off-chip memory bandwidth) are shared among multiple cores as they help in improving resource utilization during workload consolidation. However, sharing can detrimentally impact average memory access time and performance due to resource contention. Prior work has proposed partitioning of shared resources – cache [1]–[5] and bandwidth [6]–[8] – and prefetch throttling [9] to mitigate contention, reduce or hide memory access time and improve performance.

Recent work has proposed combining cache and bandwidth partitioning [10]–[12], prefetching and cache partitioning [13], [14] and bandwidth partitioning and prefetching [15], [16] to provide additional performance gains. The key insight from this work is that coordinated management of two techniques is more advantageous than considering each in isolation because of the trade-offs that are made possible. However, prior works

have not considered combining all three techniques. This misses out on several opportunities. The goal of this paper is to explore the potential of combining all three techniques and thereby facilitate inter-resource interactions and tradeoffs.

In this paper we show that coordinated management of cache partitioning, bandwidth partitioning and prefetch throttling provides the following advantages. Firstly, it makes it possible to address more applications and cover a broader range of workloads, as shown in our in-depth performance characterization (see Section II). The results show that 90% of the applications in the SPEC CPU2006 suite have performance sensitivity (over 10% change in IPC) to at least one of the techniques, and 70% are also sensitive to multiple techniques. Secondly, managing these techniques jointly opens up the opportunity to new and improved trade-offs. There are synergistic interactions between the techniques and they cannot be realized if cache partitioning, bandwidth partitioning and prefetch throttling are not jointly managed.

As an example, consider the case of a workload comprising of two applications. The first application, *lbm*, is sensitive to bandwidth and prefetching while the second, *xalancbmk*, is sensitive to cache size and has lower performance when prefetching is enabled. The best solution when managing all three techniques is to give *xalancbmk* a large cache allocation, small bandwidth allocation and disable the prefetcher, while giving *lbm* a large bandwidth allocation, small cache allocation while keeping the prefetcher active. Figure 1 shows the performance from coordinated management of all three

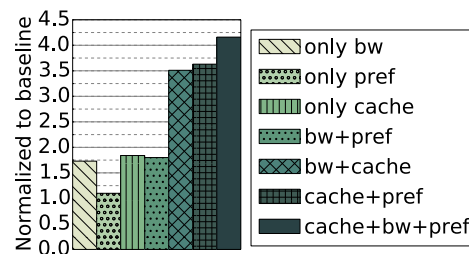


Fig. 1: Workload with *lbm* and *xalancbmk*. Total bandwidth 16 GB/s, total cache size 2MB. Executing applications for 1B instructions, more details in Section IV. Settings: *lbm*-prefetching active, 12 GB/s, *xalancbmk*- prefetcher inactive, 4 GB/s, determined from characterization (Section II). Cache partition sizes are decided dynamically.

techniques (*cache+bw+pref*) compared to managing a subset of the techniques. The results show that the solution managing all three techniques is better than others managing two of the techniques, and leads to an additional performance gain of 15%. The main challenge of coordinately managing all three techniques is the complexity of evaluating all possible allocations dynamically and determining the best possible allocation while exploiting the large number of possible trade-offs.

Guided by our characterization results, we propose CBP, a technique for dynamic and coordinated management of Cache partitioning, Bandwidth partitioning and Prefetch throttling for multi-programmed workloads. CBP consists of three local controllers, one for each resource, that together with a coordination mechanism manages and allocates the resources. CBP dynamically tunes the three local resource controllers in an iterative fashion in order to address the complexity of navigating the multi-resource search space. While we explore different coordination policies, the one that works best is the following: First, cache and bandwidth are allocated. The cache and the bandwidth allocation controllers provide a minimum allocation to all cores to ensure that requests from non memory-intensive applications do not experience significant delay because of requests from other co-running applications. The remaining cache capacity is allocated with the goal of minimizing the aggregate number of LLC misses while the remaining bandwidth is allocated based on the queuing delay. This allocation is carried out mainly based on the statistics collected from the previous interval. As a next step, the prefetch setting is determined for the current interval by sampling the impact of different prefetch settings on performance for the current allocation of bandwidth and cache. The prefetcher performance influences the next reallocation of cache space and bandwidth. The feedback mechanism between the different techniques dynamically adapts the allocations in order to reach a good configuration depending on the characteristics of the individual applications in the workload. Our approach of combining local resource controllers with a feedback mechanism reduces the computational complexity.

In summary, we make the following contributions:

(a) We present an in-depth characterization of the performance impact of cache, bandwidth and prefetching on the entire SPEC CPU2006 suite. Our characterisation results provide several insights: i) a majority of the applications (over 90%) are sensitive to one or multiple techniques, ii) managing cache, bandwidth and prefetch opens up opportunities for exploiting more trade-offs and improving performance for consolidated workloads, and iii) managing cache, bandwidth and prefetch jointly has the potential to outperform combinations of two of the techniques.

(b) We propose CBP, a technique to dynamically manage the three resources in coordination. The solution is based on simple heuristics in order to sidestep the complexity associated with evaluating all possible configurations and choosing the best performing configuration. CBP works by employing individual resource managers to determine the appropriate setting

for each resource and a coordinating mechanism to enable inter-resource trade-offs.

(c) We evaluate our solution with multi-programmed workloads on a 16-core tiled CMP. CBP improves performance by up to 36% (geom. mean 11%) compared to the state-of-the-art technique that manages cache partitioning and prefetching in a coordinated manner and by up to 86% (geom. mean 50%) compared to an S-NUCA without cache partitioning, bandwidth partitioning and prefetching.

The rest of the paper is organized as follows. Section II motivates the need for a coordinated approach using cache partitioning, bandwidth partitioning and prefetch throttling. Section III describes our proposed solution in detail. We then discuss the methodology in Section IV and Section V presents the evaluation of the proposal. We provide an overview of related work in Section VI and conclude in Section VII.

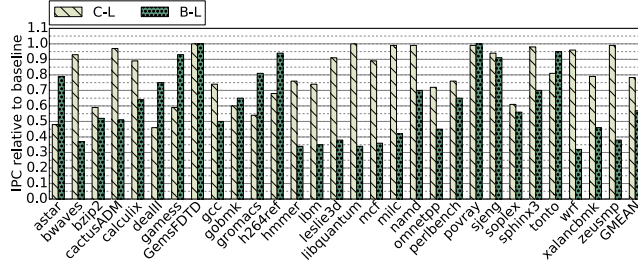
II. CHARACTERIZATION

In order to motivate the need for coordinated management of cache partitioning, bandwidth partitioning and prefetch throttling, we perform a detailed characterization study of applications in the SPEC2006 CPU suite. The aim of this study is to: i) characterize applications to determine the extent to which they are performance sensitive to cache, bandwidth and prefetch settings, ii) understand the different resource interactions, their impact on performance and the inter-resource trade-offs that are possible, and iii) demonstrate the performance potential of coordinated management of all three resources over a subset of resources.

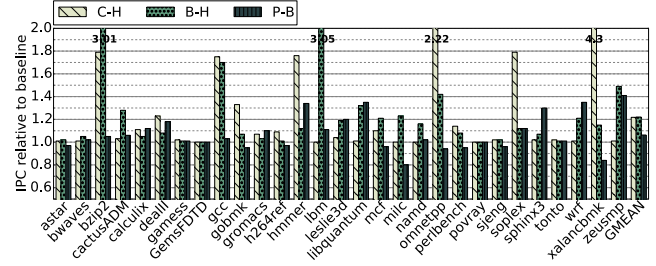
A. Sensitivity to cache, bandwidth and prefetch settings

To understand the sensitivity of applications to cache, bandwidth and prefetch settings, we model a system consisting of one out-of-order core with a 3-level cache hierarchy using the Sniper simulator [17]. Details about the methodology are provided in Section IV. For this experiment, the baseline LLC and bandwidth allocation is 512 KB and 4 GB/s, respectively. We run the application in steady-state for 1B instructions and use IPC as a measure of performance.

Figure 2 shows the performance impact of changing the cache allocation, bandwidth allocation and enabling of prefetching normalized to the baseline allocation without prefetching. Note that we only change the setting for one resource at a time. In Figure 2a, C-L and B-L represent low allocation settings where the cache allocation is decreased to 128 KB and the bandwidth allocation is decreased to 1 GB/s, respectively, while prefetching is disabled. Similarly, in Figure 2b, C-H and B-H represent high allocation settings, where the cache allocation is increased to 2MB and the bandwidth allocation is increased to 16GB/s, while prefetching is disabled. Finally, P-B represents the setting where prefetching is enabled with the baseline cache and bandwidth allocation. We classify applications as *performance sensitive* to a specific resource if the modified allocation results in a 10% deviation from the baseline IPC. We refer to applications that are performance sensitive to a change in cache allocation as cache sensitive (CS), sensitive to change in bandwidth allocation as



(a) Slowdown when decreasing the cache size to 128kB (C-L), and slowdown when decreasing the bandwidth allocation to 1GB/s (B-L) in comparison to the baseline allocation, with prefetching disabled.



(b) Performance improvement from increasing cache allocation to 2MB (C-H), and increasing bandwidth allocation to 16GB/s (B-H) in comparison to the baseline allocation, with prefetching disabled. Performance improvement from enabling prefetching for the baseline allocation (P-B).

Fig. 2: Performance impact of changing cache size, bandwidth allocation and prefetcher setting. There are 6 CS-BS-PS applications, 8 CS-BS, 6 BS-PS, 3 CS, 3 BS and 3 applications are insensitive (I) to all three techniques.

Application	CS	BS	PS	Application	CS	BS	PS
astar	✓	✓		bwaves		✓	
bzip2	✓	✓		cactusADM		✓	
calculix	✓	✓	✓	dealII	✓	✓	✓
games	✓			GemsFDTD			
gcc	✓	✓		gobmk	✓	✓	
gromacs	✓	✓		h264ref	✓		
hmmer	✓	✓	✓	lbm	✓	✓	✓
leslie3d		✓	✓	libquantum		✓	✓
mcf	✓			milc		✓	✓
namd		✓		omnetpp	✓	✓	
perlbench	✓	✓		povray			
sjeng				sphinx3	✓	✓	✓
sphinx3		✓	✓	tonto	✓	✓	
wrf		✓	✓	xalancbmk	✓	✓	✓
zeusmp		✓	✓	Count	17	23	12

TABLE I: Summary of performance sensitivity to cache, bandwidth and prefetch setting. Note that the classification of applications is based on the specific settings for cache, bandwidth and prefetch that we have evaluated.

bandwidth sensitive (BS) and sensitive to prefetch throttling as prefetch sensitive (PS). Table I summarizes the sensitivity of the applications to the three resources.

The sensitivity results for cache size show that nearly 60% of the applications (17 out of 29) are sensitive to changes in cache allocation. The extent to which applications are performance sensitive varies greatly with a performance increase of up to 4x in some cases. Furthermore, a larger number of applications are sensitive in the low allocation setting in comparison to high allocation setting (17 compared to 11). The sensitivity results for bandwidth allocation also shows a similar trend, as more applications are sensitive in the low allocation setting (23 compared to 15). Also, the extent of performance sensitivity varies greatly with an increase of up to 3x. The sensitivity results for prefetch throttling indicates that nearly 38% of the applications (10 out of 29) are sensitive to prefetching and experience a speedup. However, some applications (2 out of 29) suffer from a slowdown due to prefetching. In summary, we make the following observation:

OBSERVATION 1. In SPEC CPU2006 suite 90% of the applications are sensitive to one resource, while 70% are sensitive to multiple resources and the extent of sensitivity

varies greatly.

B. Inter-resource interactions and trade-offs

Next, we investigate inter-resource interactions and trade-offs that are enabled when jointly managing cache, bandwidth and prefetching. We focus on intra-application resource interaction initially. There are four possible types of interactions within an application: cache-bandwidth-prefetch, bandwidth - prefetch, cache - prefetch and cache - bandwidth.

Regarding the cache-bandwidth-prefetch trade-off, we want to find out how the performance impact from prefetching varies with the allocation of cache and bandwidth. Figure 3 shows the performance impact of prefetching for three different cache/bandwidth settings normalized to the respective baseline setting without prefetching. The cache and bandwidth setting for an application in a low allocation scenario (P-L) is 128KB and 1GB/s, the baseline allocation scenario (P-B) setting is 512KB and 4GB/s while the high allocation scenario (P-H) setting is 2MB and 16GB/s.

For some applications, lower bandwidth and cache allocation leads to higher sensitivity for prefetching as seen in *hmmer*. This is because avoiding a miss altogether, as a consequence of accurate prefetching, can have a larger impact in low allocation settings where the bandwidth is scarce and the memory queuing delays tend to be longer. Also, there are applications like *gcc* which experience higher prefetch sensitivity with a larger cache and bandwidth allocation. The

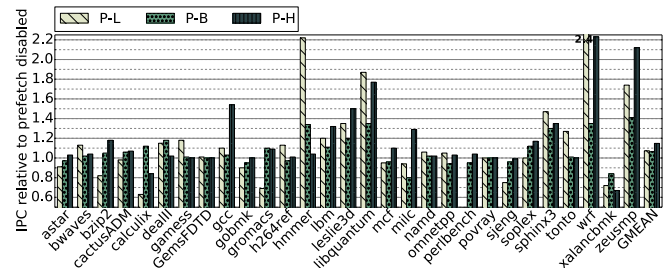


Fig. 3: Performance impact of enabling prefetching relative to allocation of cache and bandwidth, for allocation settings; L:128kB,1GB/s B:512kB,4GB/s H:2MB,16GB/s

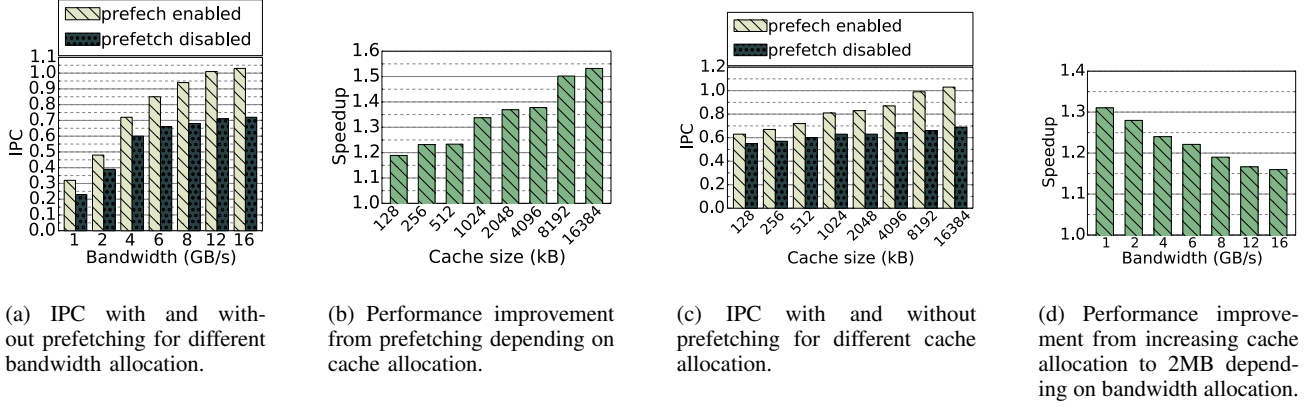


Fig. 4: leslie3d - example of interactions and its impact on performance within a single application.

results indicate that applications tend to be prefetch sensitive in some settings and prefetch insensitive in others. We make the following observation:

OBSERVATION 2. *Allocation of cache and bandwidth influences prefetch sensitivity. Furthermore, applications tend to be prefetch sensitive in some settings and prefetch insensitive in others.*

In the interest of space we use *leslie3d* as a representative example to illustrate the other pairwise resource interactions and trade-offs, since it is sensitive to all three techniques. Note that the baseline setting will be used for the resources unless specified otherwise. The bandwidth - prefetch interaction manifests in two different ways. Firstly, prefetching typically increases the number of memory accesses and this in turn increases the bandwidth pressure. In the case of *leslie3d* prefetching results in a 15% increase in the number of memory requests in comparison to the baseline. Note that prefetch misses, i.e. prefetched blocks that are evicted before use, can result in a further increase in pressure on the memory bandwidth. Secondly, the performance improvement from prefetching can be influenced by the bandwidth allocation. The results in Figure 4a show the performance for different bandwidth allocations with and without prefetching. We make the following observation:

OBSERVATION 3. *A larger bandwidth allocation can compensate for increased bandwidth demands, due to inaccurate prefetches, leading to increased performance with prefetching.*

The cache - prefetch interaction also manifests in two main ways. Firstly, the performance loss from reduced cache allocation can be offset if prefetching is effective. Figure 4c shows the IPC for different cache allocations with and without prefetching. The results show that the performance of a 128KB allocation with prefetching is better than a 512KB allocation without prefetching. Secondly, larger cache sizes (if it is used efficiently) can lead to higher speedup from prefetching. Figure 4b shows the performance improvement from prefetching with different cache allocations normalised to the respective cache allocation without prefetching. The results show that prefetching is effective with lower cache allocation and that

its effectiveness can increase with additional allocation. The reason for this behaviour is that a larger cache reduces the number of memory accesses which has the same effect as increasing the available bandwidth, i.e. a lower queuing delay, which is more forgiving when there is an increase in memory accesses caused by inaccurate prefetches (in *leslie3d* there is a 15% increase in dram accesses caused by prefetching). These results lead to the following observation:

OBSERVATION 4. *A trade-off can be made between either increasing cache size or enabling prefetching, leading to the same performance, for applications which are performance sensitive to both cache and prefetching.*

As for the cache - bandwidth interaction, a lower bandwidth allocation can result in a larger sensitivity to cache size. Figure 4d shows the performance improvement from increasing the cache allocation from 512kB to 2MB with different bandwidth allocation settings. The results show that performance improvement from additional cache allocation is much higher in low bandwidth allocation settings (see the result for 1GB/s bandwidth allocation). This is because the average cost of a miss is much higher in the case of lower bandwidth allocation. The results also show that a large cache allocation can reduce the performance sensitivity to bandwidth allocation (see the result for 16GB bandwidth allocation). These results lead to the following observation:

OBSERVATION 5. *A trade-off can be made between either increased cache space or increased bandwidth allocation, for applications which are performance sensitive to both cache and bandwidth.*

We now describe how the observed intra-application interactions and trade-offs can be leveraged in the inter-application setting for multi-programmed workloads. Let us revisit the example of running a simple workload comprising two application (*lbm* and *xalancbmk*) on a dual-core system with 2MB LLC capacity and 16GB/s bandwidth, discussed in Figure 1. For achieving the best aggregate performance we expect *xalancbmk* to get the majority of the cache (nearly 1.75MB), while *lbm* is given a smaller cache allocation of 256KB. For bandwidth, we would expect *lbm* to have a large allocation (12GB/s) of the available bandwidth and *xalancbmk* to get a

smaller allocation (4GB/s). This is reflected in Observation 5 about the trade-off between cache and bandwidth where we would prioritize the application that shows the highest sensitivity for the resource. Furthermore, as reflected in Observation 2, we expect prefetching to be more effective for *lbm* since it has a large allocation of bandwidth. In the case of *xalancbmk*, prefetching leads to lower performance regardless of the allocation of cache and bandwidth.

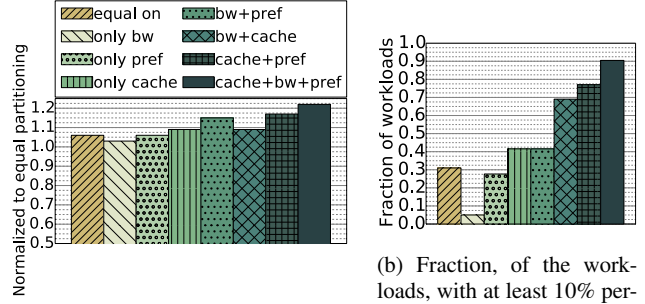
C. Generalizability of performance characterization

The results in the performance characterization are based on a range of configurations for cache (128KB up to 16MB) and for bandwidth (1GB/s up to 16GB/s). This range of explored configurations represents common design points in existing multicore chips. We do not expect that a wider range of configurations to fundamentally change the observations. We investigate the impact of using two other prefetchers, GHB [18] and AMPM [19]. The average performance impact from prefetching increases from 6% using the stride prefetcher to 10% with GHB and to 19% with the AMPM prefetcher. With the AMPM prefetcher 16 applications are positively performance sensitive compared to 10 applications with the stride prefetcher. We show the impact of using these prefetchers with CBP in Section V-A. The results show that using an aggressive prefetcher increases the performance sensitivity and the potential for coordinated management as well.

D. Potential for coordinated management

In order to show the potential for coordinated management of cache, bandwidth and prefetch throttling we run 640 randomly generated workloads each comprising 4 SPEC 2006 CPU applications. We compare the performance of jointly managing all the three resources to other resource managers that only manage a subset of these resources. For this experiment the baseline allocation of cache and bandwidth for each application is 512kB and 4GB/s. We use an exhaustive search algorithm to find the best static configuration (over 1B instructions) for the different resources when running each workload. Figure 5a shows average (geometric mean) performance with different resource managers normalized to the baseline settings without prefetching. *equal on*, depicts the performance when prefetching is enabled for all applications and improves performance by 6% while *only pref*, depicts the performance when prefetching is selectively activated and improves performance by 9%. *cache+bw+pref* results show that coordinately managing cache partitioning, bandwidth partitioning and prefetch throttling improves performance by 5% compared to the best combination of two techniques (22% compared to 17%).

Figure 5b shows the number of workloads (among the 640 workloads considered) that experience a performance gain of at least 10% using the different resource managers discussed previously. The results show that 90% (597) of the workloads are sensitive to the resource manager that jointly manages all three techniques. A smaller fraction of the workloads are sensitive to resource managers that manage a subset of these



(a) Performance potential of using different resource managers.

Fig. 5: Potential for coordinated management measured using 640 random workloads of 4 SPEC CPU2006 applications. Performance is obtained using an exhaustive search algorithm that evaluates bandwidth settings (2GB/s, 4GB/s, 6GB/s), cache settings (256kB, 512kB, 1024kB) and prefetching settings (active/inactive), in conjunction, to determine the resource allocation for each application in the workload that maximizes aggregate performance.

techniques (77% are sensitive to *cache+pref* resource manager and 69% are sensitive to the *cache+bw* resource manager).

In summary, the results from the characterization study demonstrate that around 90% of applications in the SPEC 2006 CPU suite are sensitive to different resources and that coordinately managing them opens up new possibilities for improving performance by trading resource allocations. Furthermore, jointly managing these resources has the potential to cover a broader range of workloads and outperform resource managers that manage a subset of resources. In the next section we will discuss how the proposed resource manager, CBP, determines cache, bandwidth and prefetch settings for different applications in a workload.

III. CBP RESOURCE MANAGER

Section III-A provides an overview of the CBP resource manager. Section III-B discusses the individual resource controllers while Section III-C describes how the coordination mechanism ties the local resource controllers together. Finally, the implementation and overhead of the proposed mechanism is discussed in Section III-D.

A. Overview

CBP is a coordinated mechanism for dynamically managing cache partitioning, bandwidth partitioning and prefetch throttling. The design consists of one local controller for each of the three techniques, and a coordination mechanism, as shown in Figure 6.

The cache allocation controller estimates the number of misses for different cache sizes using auxiliary tag directories (ATDs) [1] and uses this as input for determining cache allocation. The cache allocation per application is determined such that it reduces the aggregate number of cache misses for the entire workload. The bandwidth allocation controller uses memory request queuing delay experienced by the applications as input and allocates the available bandwidth in proportion

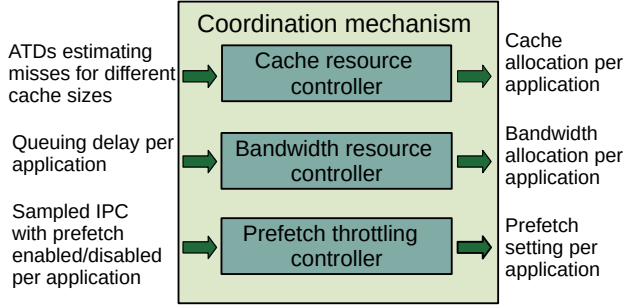


Fig. 6: Overview of CBP resource manager.

to the delay. The bandwidth allocation controller, assigns a larger allocation of the available bandwidth to applications that experience longer queuing delay, and a comparatively lower allocation to those that experience shorter queuing delays. Note that the cache and the bandwidth allocation controllers provide a minimum allocation to all applications to ensure that requests from applications that are not memory-intensive do not experience significant delay because of requests from other co-running applications. Lastly, the prefetch controller samples IPC with and without prefetching to determine whether the prefetcher should be enabled/disabled for each application.

The three techniques are dynamically tuned using the coordination mechanism in an iterative manner such that the local controller takes into consideration the decisions taken by the other controllers.

B. Local resource allocation controllers

A partitioning solution for shared resources like cache and bandwidth typically comprises two components: allocation policy, that determines how a resource is divided among multiple co-running applications, and an enforcement mechanism, that enforces the partitioning decision. Similarly, prefetch throttling involves a policy to determine the best prefetch setting to use and a mechanism implemented in hardware to enforce the setting. In the context of CBP, the policy component is of particular interest because it enables inter-resource trade-offs. We discuss the allocation policy in this section and defer the details of the enforcement mechanism to Section III-D.

1) *Cache partitioning*: The cache allocation controller uses the Lookahead algorithm [1], to determine cache allocation. In a nutshell, the algorithm computes the utility for each application where utility is the measure of how many additional misses can be reduced with allocation of cache ways. It then computes the number of ways that maximizes the utility for each application (while ensuring this is less than the total number of ways available for allocation). Finally, it compares the utility values for the different applications, determines the application that has the highest utility and assigns the pre-computed number of ways that maximizes the utility for that application. The process repeats, with recomputation of utility for each application and reassignment of available cache ways to the application that has the largest utility, until the rest of

Algorithm 1: Bandwidth allocation controller pseudo-code

Input : A list *queuingDelayPerApplication*
Output: A list *bandwidthAllocationPerApplication*

```

1 At time period reconfiguration_interval;
2 remainingBandwidth = (totalBandwidth - min_bandwidth_allocation * totalNumberOfCores);
   totalDelay = 0;
3 for i ← 0 to totalNumberOfCores - 1 do
4   totalDelay += queuingDelayPerApplication[i];
5   bandwidthAllocationPerApplication[i] = min_bandwidth_allocation;
6 end
7 for i ← 0 to totalNumberOfCores - 1 do
8   bandwidthAllocationPerApplication[i] += (queuingDelayPerApplication[i] / totalDelay) * remainingBandwidth;
9 end

```

the available capacity is distributed. The allocation controller relies on sampled ATDs to estimate, based on past behaviour, the number of misses that can be avoided with additional allocation of cache ways for each application. In order to adapt to an inclusive cache hierarchy, we assign a minimum allocation of cache space (*min_ways*) to all the applications before distributing the remaining capacity.

2) *Bandwidth partitioning*: We propose a bandwidth allocation algorithm, that partitions bandwidth proportional to the memory queuing delay experienced by each application. The pseudo-code for the proposed bandwidth allocation controller is outlined in Algorithm 1. The controller assigns a minimum bandwidth allocation (*min_bandwidth_allocation*) for each application. The minimum allocation is important because it ensures that the less memory intensive applications have a low memory latency. A sensitivity study for the minimum bandwidth parameter is presented in Section V-C3. The remaining bandwidth is set for distribution among the applications (see line 2). The algorithm computes the total queuing delay by summing up the individual queuing delays experienced by each application (line 4) while assigning the minimum allocation to each application (line 5). As the next step, the remaining bandwidth is allocated proportionally to the queuing delay experienced by the application (line 7-9). For precision we need performance counters for monitoring queuing delays similar to the performance counters for L2 stall cycles (PMU event “STALLS L2 PENDING”) which are available in Intel processors [20].

3) *Prefetch throttling*: The prefetch throttling policy determines the best prefetcher settings for each application. The pseudo-code for the prefetch throttling controller is outlined in Algorithm 2. The algorithm considers two possible settings – the prefetcher enabled vs. disabled – but can easily be extended to support other aggressiveness settings as well. The algorithm uses the sampled IPC values, for each application obtained, with different prefetcher settings over a sample period (*prefetch_sampling_period*) as input. The algorithm first computes the speedup from prefetching for each application using the sampled IPC values. If the speedup is below a threshold (*speedup_threshold*) the prefetcher is deactivated for the next prefetch interval (*prefetch_interval*) (line 3-4). If

Algorithm 2: Prefetch throttling controller pseudo-code

Input : Two lists *ipcWithPrefetchingActive*,
ipcWithPrefetchingInactive

Output: A list *prefetchSettingPerCore*

```

1 At time period prefetch_interval;
2 for  $i \leftarrow 0$  to totalNumberOfCores-1 do
3   if (ipcWithPrefetchingActive[i]/
        ipcWithPrefetchingInactive[i] >
        speedup_threshold then
4     | prefetchSettingPerCore[i] = 0;
5   else
6     | prefetchSettingPerCore[i] = 1;
7   end
8 end

```

the speedup is above the threshold prefetching is activated for the next prefetch interval (line 6). The prefetch-throttling controller is generic enough to support any type of prefetcher.

C. Coordination Mechanism

The goal of the coordination mechanism is to ensure that each local controller takes into account the decisions taken by other controllers. This is necessary to exploit the trade-offs outlined earlier in Section II. There are two essential tasks carried out in order to establish this: i) inter-controller interaction ii) controller ordering.

Inter-controller interaction: Figure 7 provides an overview of the interactions between the different resource allocation controllers. Firstly, we describe how the bandwidth allocation controller decisions takes into account the decisions made by the cache and the prefetch controller. The bandwidth allocation controller, makes decisions based on the queuing delay of each application which is affected by the number of memory accesses. The cache allocation controller through a larger cache allocation can reduce the number of memory accesses (Interaction #1). This leads to a lower bandwidth allocation for applications that can efficiently use the cache. The prefetch throttling controller influences the bandwidth allocation decision mainly through prefetch misses (prefetched data that is not used) (Interaction #2). This is because prefetch misses lead to more memory requests and potentially a higher queuing delay.

Next, we describe how the prefetch throttling controller takes into account the decisions made by the other controllers. The prefetch-throttling controller makes decisions by sampling the IPC with different prefetcher settings over a specific interval. The sampled IPC values, used to determine the prefetcher setting, reflects the effect of cache and bandwidth allocation decisions made by the respective resource

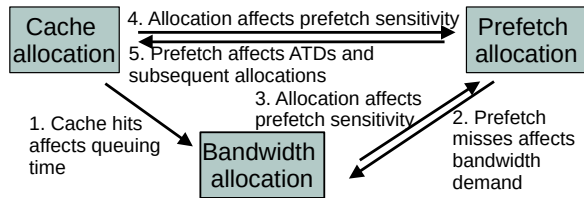


Fig. 7: Interactions among the different resource allocation controllers in CBP .

controllers (Interaction #3-4). Finally, we describe how the cache allocation controller is affected by the prefetch throttling controller (Interaction #5). If an application benefits from prefetching, this reflects on the miss count values monitored in the ATDs. Since the cache allocation is computed based on the counter values observed in the ATD, this ends up affecting the subsequent cache allocation decision, resulting in a smaller cache allocation for prefetch sensitive applications.

Section V-A evaluates alternate designs that only exploit a subset of the interactions discussed above to understand the performance impact of failing to exploit the different inter-resource interactions and trade-offs.

Controller ordering: Since the decision taken by one controller has the potential to influence those taken by others, the order in which local controllers make allocation decisions is important. We discuss the timeline showing when the different resource allocation controllers are invoked and how they interact with each other, in an iterative manner, using the example illustrated in Figure 8. The three resource controllers, are invoked after a specific interval that we refer to as the *reconfiguration_interval* in the sequence shown in the figure.

First cache and bandwidth are equally partitioned among all applications at time 0, since information about misses and queuing delays is initially unavailable, as shown in Step 0. This is followed by sampling the IPC of applications with different prefetch settings for a specific interval (twice the *prefetch_sampling_period*) as shown in Step 1. Note that the prefetch controller is ordered (invoked) after cache and bandwidth allocation to exploit the observation that the prefetch effectiveness is determined by the current allocation of cache and bandwidth. Based on the sampled IPCs the prefetch throttling controller determines the appropriate prefetcher setting for each prefetcher for the current *reconfiguration_interval*.

Cache and bandwidth allocation controllers are again invoked after the *reconfiguration_interval*, as shown in Step 2. Since the allocation of cache and bandwidth are based on statistics collected during the previous interval, the relative ordering among them has little impact. A cache allocation decision for the next interval (shown in step 2.1) is influenced by the number of hits and misses observed in the previous interval. The ATD values will be halved after each reconfiguration, in order to be sensitive to changes in the last time interval while the per application queuing delays are accumulated with those from the previous interval. The bandwidth

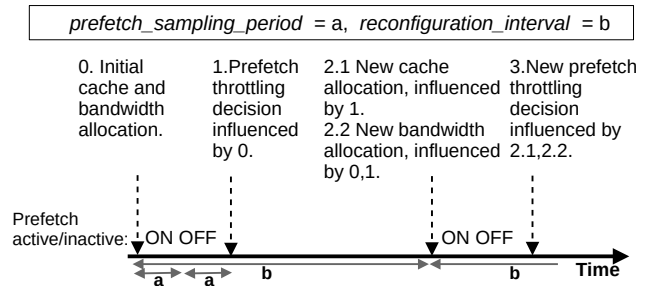


Fig. 8: Timing and interactions of CBP resource manager.

allocation decision shown in Step 2.2, is influenced both by the previous bandwidth and cache allocation and prefetcher setting in the previous interval as discussed previously. Finally, the prefetcher throttling controller shown in Step 3 is influenced by the new cache and bandwidth allocation. The impact of using a different order between prefetch, cache and bandwidth is evaluated in Section V-A. The interactions among the different resource allocation controllers take place over multiple iterations, and are key to finding an effective solution.

D. Implementation

The computational overhead of CBP resource management is low since the design uses heuristics to guide the allocation decisions instead of exhaustively evaluating the different possible allocations.

The cache allocation controller needs hardware support in order to estimate the number of misses with different cache sizes. We use sampled ATDs [1] as discussed previously to compute the effect of different cache allocations on the misses. When enforcing cache partitioning there is an overhead associated with invalidations due to reconfiguration decisions. This is modelled faithfully by invalidating the addresses and re-fetching them when accessed, this includes the latency and impact on bandwidth from accessing memory. We have used the enforcement mechanism proposed by Holtryd et al. [21] since it is suitable for a modern tile-based CMP and is both fine-grained and locality-aware. The enforcement mechanism uses per-core Cache Bank Tables (CBTs), where mappings between addresses and banks are recorded. When a request needs to access the LLC, the CBT is used to identify the cache bank that the address is mapped to. Inside each bank, way partitioning hardware divides the capacity. The enforcement results in a partition granularity of 32kB on our system, see Section IV. We incur hardware cost for implementing ATDs and cache partition enforcement [1], [21].

The bandwidth partition enforcement is done in likeness with Intel Memory Bandwidth Allocation (MBA) technology [22], [23] which is commercially available. The solution uses delays as a way to allocate the bandwidth. An application with a high delay has a low allocation, and experiences a longer queuing delay for each memory access. In our solution the additional delay is added after the LLC, instead of after the L2, as in the original proposal.

The overhead of prefetch-throttling comes from sampling an application with different prefetcher settings. This is because deactivating prefetching for an application can be detrimental for its performance, especially when prefetching is effective. Likewise, it is detrimental to turn it on prefetching (for a sample period) for an application whose performance is hurt by prefetching.

IV. EXPERIMENTAL METHODOLOGY

A. Simulated Architecture

We evaluate our proposal on a 16-core tiled CMP architecture modeled using the Sniper Simulator [17]. Each tile has an out-of-order (OOO) core with a private L1 data and

instruction cache, a unified private L2 cache and an LLC bank of 512KB. The cache latencies assumed have been modelled using CACTI 6.5 [24]. Details about the baseline architecture are shown in Table II. A sensitivity study is provided for the CBP parameters in Section V-C.

B. Methodology

We use the entire SPEC CPU2006 suite in our evaluation. The applications are in the format of whole program pinballs [25]. We create 14 workload mixes (each comprising 16 applications) by randomly selecting applications from the entire SPEC CPU2006 suite. Details about workload mixes are presented in Table III.

We fast-forward for 16B instructions (in total) and then carry out detailed simulation until all benchmarks have completed at least 500M instructions. Statistics are reported based on the detailed simulation of 500M instructions. After this period the applications continue to run and compete for resources to avoid having a lighter load on long running applications. The methodology is in line with earlier works [4], [5], [26].

C. Metrics

We report normalized weighted speedup over baseline for each workload. This is computed by $\frac{1}{N} \sum_{i=1}^N \frac{IPC_{i, RM}}{IPC_{i, baseline}}$, in order to evaluate system performance for multi-programmed workloads. RM refers to the system with a resource manager that manages cache, bandwidth and prefetcher settings and the baseline refers to a system with unpartitioned cache and bandwidth and without prefetching.

We also report average normalized turnaround time (ANTT) for each workload since this is a user-oriented performance metric which shows fairness. ANTT is given by $\frac{1}{N} \sum_{i=1}^N \frac{CPI_{i, RM}}{CPI_{i, baseline}}$.

Table IV shows the resource managers we evaluate, in addition to CBP, and the corresponding settings they use for cache, bandwidth and prefetching. The baseline configuration represents a system with unpartitioned cache, unpartitioned

Cores	16 cores, x86-64 ISA, 4GHz, OOO, 128 ROB entries, dispatch width 4
L1 caches	32KB, 8-way set-associative, split D/I, 1-cycle latency
L2 caches	128KB private per-core, 8-way set-associative, inclusive, 6-cycle data and 2-cycle tag latency
LLC	512KB per-tile, 16-way set-associative, inclusive, 9-cycle data and 2-cycle tag latency, LRU
Coherence protocol	MESIF-protocol, 64 B lines, in-cache directory
Global NoC	4x4 mesh, 4-cycles hop latency (3-cycle pipelined routers, 1-cycle links)
Memory controllers	4 MCUs, 1 channel/MCU, latency 80 ns, 16GB/s per channel
Prefetcher	stride-based, located in L2, 4 prefetches stop at page boundary, 8 flows/core
Alternative prefetchers	GHB Distance prefetching(G/DC), width=4, depth=1, GHB =512, Index table = 512 AMPM, 256 pages, degree 4
CBP parameters	<i>reconfiguration_interval</i> =10ms <i>prefetch_sampling_period</i> =0.5ms, <i>speedup_threshold</i> = 1.05, <i>prefetch_interval</i> =10ms, <i>min_bandwidth_allocation</i> =1, <i>min_ways</i> =4

TABLE II: Configuration of the simulated 16-core tiled CMP.

w#	Types	Benchmarks
w1	4CS-BS-PS,5CS-BS,3BS-PS,3CS,1BS	xalancbmk(xa),gromacs(gr),libquantum(li)(2),h264ref(h2),zeusmp(ze),tonto(to),soplex(so),lbml(lb),perlbenc(pe),calculix(ca),milc(mi),sphinx3(sp),bwaves(bw),gobmk(go),gamses(ga)
w2	3CS-BS-PS,5CS-BS,5BS-PS,2CS,1BS	lb,to,pe,go,gcc(gc),mi,li(2),namd(na),h2,cactusADM(cac),ze(2),ca,so,astar(as)
w3	6BS-PS,CS,5BS,4I	bw(2),povray(po)(2),sjeng(2),sp(2),na(2),ze,GemsFDTD(Ge),cac,li,mi,wrf(wr)
w4	CS-BS-PS,2CS-BS,5BS-PS,3CS,2BS,3I	po,bw(2),h2,sjeng(sj),li(2),gr,na,mi(2),as,Ge,ga,wr,lb
w5	5CS-BS-PS,10CS-BS,BS-PS	deallI(de),omnetpp(om)(2),go(2),hmmmer(hm),xaleslie3d(le),bzip2(bz)(2),gc,so,mcf(mc),pe,ca(2)
w6	3CS-BS-PS,5CS-BS,4BS-PS,2CS,2BS	sp,bw(2),h2,om,li,gr,go,mi(2),as,hm,ga,le,lb,ca
w7	2CS-BS-PS,2CS-BS,3BS-PS,5CS,4I	po(2),to,sj,h2(2),na,lb(2),ze(2),gr,Ge,as,wr,ga
w8	4CS-BS-PS,4CS-BS,2CS-PS,3BS-PS,3BS	de,bw(3),xa,mi(3),om,li(2),bz,go,so,hm,pe
w9	2CS-BS-PS,5CS-BS,2BS-PS,3CS,BS,2I	gc,po,to,hm,sj,h2,bz,ze,gr,so,Ge,as,pe,wr,ga,cac
w10	2CS-BS-PS,3CS-BS,6BS-PS,CS,2BS,2I	sj,bw(2),de,na,li(2),om,ze,mi(2),xa,Ge,bz,wr,gc
w11	2CS-BS-PS,4CS-BS,4BS-PS,CS,2BS,3I	po,om,sj,go,na(2),le,ze,xa,Ge,bz,wr,ca,sj,sp,gc
w12	6CS-BS-PS,8CS-BS,2CS	de,to,go,h2(2),hm,gr,xa,as(2),bz,ga,gc,lb,so,ga
w13	3CS-BS-PS,2CS-BS,4BS-PS,4CS,3I	to,po,h2,sj,gr,na,as,ze,ga,Ge,lb(2),li,to,mi,wr
w14	5CS-BS-PS,2CS-BS,5BS-PS,CS,BS,2I	de,bw,go,po,hm,na,xa,ze,so,Ge,mc,li,pe,mi,ca,wr

TABLE III: 16-core workload.

bandwidth and with prefetching disabled. *equal off* configuration represents a system where cache and bandwidth are equally partitioned and prefetching is disabled. *only cache* represents the configuration where cache is partitioned as described in Section III-B1, while bandwidth is unpartitioned and with prefetching disabled. Likewise, *only bw* represents the configuration where bandwidth is partitioned as described in Section III-B2 while the cache is unpartitioned and with prefetching disabled. As for *only pref*, prefetch throttling is performed as described in Section III-B3 while cache and bandwidth remains unpartitioned. The resource managers that jointly manage two out of the three resources (*bw+perf*, *bw+cache*, *cache+perf*) in a coordinated manner can leverage a subset of the interactions described in Section III-C. We also compare against *CPpf* [13], a recently proposed technique for jointly managing prefetching and cache partitioning. In *CPpf*, prefetch friendly applications are allocated small partition sizes (because the benefit from prefetching can offset the performance drop from small allocation) while the rest of the cache is allocated to the non prefetch friendly applications. In our implementation, we give minimum allocation to the prefetch friendly applications and use UCP (see Section III-B1), to partition the remaining capacity among the non prefetch friendly applications. We use UCP and per application partitioning, in order to not put *CPpf* at a disadvantage in comparison to other schemes. Finally, CBP jointly manages all the three techniques dynamically.

V. EVALUATION

We first compare CBP to other resource managers that manage a subset of resources. Next, we investigate the performance impact of changing coordination, ordering and prefetcher. Finally, we carry out sensitivity analysis for the different design parameters, to understand its impact on the performance of CBP.

A. CBP Performance Analysis

Figure 9 shows normalized weighted speedup for each of the 14 workloads with the bars representing the different resource

RM	cache	bandwidth	prefetch
baseline	unpartitioned	unpartitioned	disabled
equal off	equal	equal	disabled
only cache	dynamic (see 3.2.1)	unpartitioned	disabled
only bw	unpartitioned	dynamic (see 3.2.2)	disabled
only pref	unpartitioned	unpartitioned	dynamic (see 3.2.3)
bw+pref	unpartitioned	dynamic	dynamic
bw+cache	dynamic	dynamic	disabled
cache+pref	dynamic	unpartitioned	dynamic
CPpf	dynamic	unpartitioned	enabled
CBP	dynamic	dynamic	dynamic

TABLE IV: The settings for cache, bandwidth and prefetch in the evaluated configurations.

managers. We use normalized weighted speedup as a measure of performance for the entire workload. *equal off* improves performance in 12 of the mixes and improves performance by 10% on average over the baseline. *only bw* improves performance in 7 of the mixes and on average by 4% over the baseline. *only pref* improves performance for 12 workloads and provides an average improvement of 9%. *only cache* improves performance for all the workloads and provides an average improvement of around 28%.

Coordinated management of bandwidth partitioning and prefetch throttling (*bw+pref*) leads to higher performance in comparison to the baseline in 12 workloads and an average overall improvement of 10%. Coordinated management of bandwidth and cache partitioning (*cache+bw*) improves performance across all workloads and provides an average performance improvement of 37% (up to 64%). Coordinated cache partitioning and prefetch throttling (*cache+pref*) improves performance across all workloads on average by 39% (up to 57%). *CPpf*, cache partitioning influenced by prefetching, improves performance by 39% (up to 63%).

Among the resource managers that perform coordinated management of two resources, *cache+pref* and *CPpf* achieve the best performance. The results also show that the im-

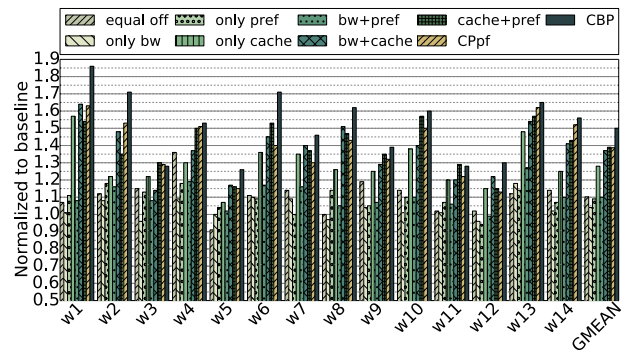


Fig. 9: Performance results, shows normalized weighted speedup over baseline.

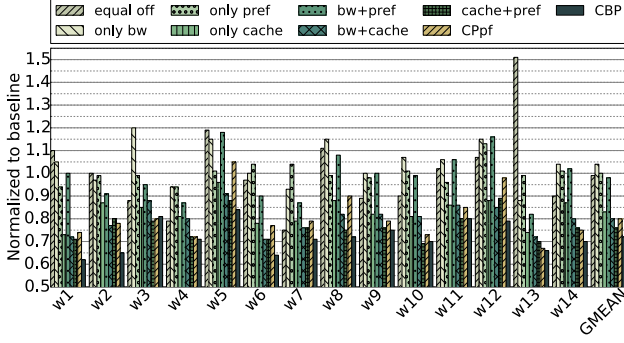


Fig. 10: Fairness results, shows average normalized turnaround time (ANTT) over baseline, where lower is better.

provement achieved with coordinated management of two techniques is larger than summing up the improvements from individual techniques. This shows that coordinated management helps exploit synergistic interactions among the different techniques, which cannot otherwise be leveraged.

Finally, CBP, outperforms previous schemes. CBP turns out to be the best performing coordinated resource manager in 14 of the 15 workloads and provides an average improvement of 50% (up to 86%). CBP improves performance by an additional 11% in comparison to the best performing resource manager that does coordinated management of two techniques, as well as state-of-the-art. In one workload, w3, CBP achieves slightly lower performance (2%) in comparison to *cache+pref*. This is because bandwidth partitioning is not very effective for this specific workload.

Figure 10 shows the average normalized turnaround time which shows the fairness of the different resource managers. Note that a lower value signifies greater fairness. On average, CBP shows 27% better fairness than the baseline and 4% better fairness than the best combination of two techniques, *cache+pref*. *cache+pref* has 4% better fairness than *CPpf*.

Case study: We investigate the performance of a single workload in detail to understand how CBP improves performance in comparison to resource managers that manage a subset of the techniques. Figure 11 shows the IPC for individual applications in a specific workload (w2) normalized to the baseline IPC. We have classified the applications in this workload into two groups. Group 1 comprises applications, from *lbm* to *gcc* (in the figure), for which the *cache+pref*

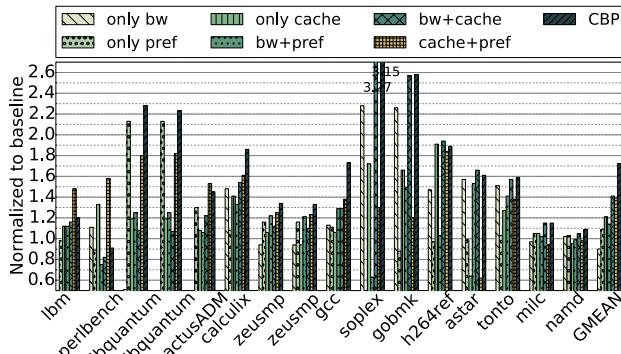


Fig. 11: Results for workload 2.

resource manager performs better than the *bw+cache* resource manager. Group 2 comprises the rest of the applications in the workload, from *soplex* to *namd*, where the *bw+cache* resource manager performs better than the *cache+pref* resource manager. *cache+pref* resource manager provides the best performance for applications in group 1 because the applications are comparatively more memory intensive and get a larger share of the available bandwidth using this resource manager. Applications in group 2 benefit from bandwidth partitioning since they then get a fair bandwidth share, and in addition are not sensitive to prefetching. When we perform coordinated management of all the three resources, we would ideally prefer to have allocation decisions made by *cache+pref* resource manager for applications in group 1 and *bw+cache* resource manager for applications in group 2. With CBP, some applications in group 1 end up with a lower allocation of bandwidth (compared to *cache+pref*) which hurts their performance (see *lbm*, *perlbenc*, *cactusADM*) while the rest of the applications in the group see a performance improvement from getting the right amount of the allocation. For the applications in group 2, CBP manages to match the performance of *bw+cache* resource manager. In summary, CBP enables better trade-offs, resulting in a solution that improves overall performance for the workload and outperforms other resource managers that only manage a subset of the techniques.

B. Performance impact of changing coordination, ordering and prefetcher

Impact of coordination: Figure 12 shows the performance impact of coordination and ordering by comparing CBP to resource managers that exploit only a subset of the interactions of CBP, and to a variation of CBP that uses a different ordering. First, we compare the performance of CBP with another resource manager, *RM1*, that uses an accuracy-based prefetch-throttling policy. Note that *RM1* coordinately manages cache and bandwidth and exploits the rest of the inter-resource interactions as CBP except those with prefetching. The lack of prefetch interaction is due to the design of the prefetch throttling mechanism that determines the prefetch setting based on the number of accurate prefetches without being influenced by the cache and bandwidth allocation. Using

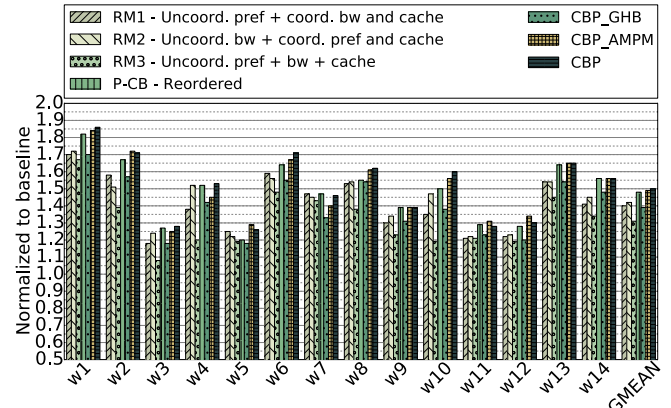


Fig. 12: Performance impact of changing coordination, ordering and prefetcher.

RM1 results in a 7.5% slowdown compared to CBP since it does not exploit bandwidth-prefetch and cache-prefetch interaction. Next, we compare against a different resource manager, *RM2*, that uses a new bandwidth allocation policy. This bandwidth allocation policy only considers the number of demand requests when determining the bandwidth allocations without considering prefetch requests. Cache partitioning and prefetch throttling is performed in a coordinated manner as in CBP. *RM2* leads to a 5.5% slowdown since it does not exploit the interaction between prefetch and bandwidth. We then combine these two policies for bandwidth and prefetch with cache partitioning, which represents an uncoordinated resource manager, *RM3*. *RM3*, leads to a performance decrease of 15% compared to CBP, since it does not exploit a majority of the interactions between cache, bandwidth and prefetch. These results show the importance of exploiting the different inter-resource interactions and the benefit of coordination in the design of CBP.

Impact of ordering: Secondly, we investigate the impact of ordering. In CBP, cache and bandwidth are first allocated and then the prefetch setting is determined based on sampling for the current allocation. In *P-CB* prefetch sampling is first performed followed by the new allocation decision for cache and bandwidth. This leads to 2% slowdown since the decision about the prefetch setting is based on the previous allocation. The impact of changing the order is limited since the reconfigurations are performed frequently (every 10ms).

Impact of prefetcher: Lastly, we investigate the impact of changing the type of prefetcher. Using the GHB prefetcher, *CBP_GHB*, leads to a speedup of $1.47\times$ over baseline, while using the AMPM prefetcher (*CBP_AMPM*) leads to a speedup of $1.49\times$. Although the single application performance is better with the more aggressive prefetchers, the bandwidth consumption is higher, which has a negative impact on the overall workload performance. This results in nearly the same performance as with the simpler stride prefetcher.

C. Sensitivity analysis

We investigate the sensitivity of CBP to different design parameters in this section.

1) *Impact of reconfiguration interval:* The reconfiguration interval, determines how frequently the different resource allocation controllers are invoked when running a workload. We investigate the sensitivity of CBP to different reconfiguration interval values, in order to determine an appropriate interval. Figure 13a shows the average (geo. mean) performance when using three different reconfiguration intervals - 1ms, 10ms, 100ms and 1000ms. A shorter reconfiguration period has the potential to adapt faster to phase change behaviour. However, it also incurs a higher overhead of invoking the local controllers. The results show that using a reconfiguration period of 1s leads to a 20% slowdown compared to using 10ms, which demonstrates the importance of frequent adaptation. Overall, the results show that using a 10ms period provides a good trade-off between quick adaptation and the overhead incurred for IPC sampling.

2) *Impact of cache size:* The results thus far assume that each tile has a baseline cache allocation of 512KB which leads to a total LLC capacity of 8MB for a 16-core CMP. We next study the impact of changing the cache capacity available for a single tile to 1MB and 2MB. Figure 13b shows the average performance achieved using CBP with different per-tile capacity normalized to the baseline configuration with the same capacity. The results show that increasing the total LLC capacity to 16MB leads to a performance improvement of 43% over baseline while increasing the cache size to 32MB leads to a performance improvement of 41%.

3) *Impact of changing bandwidth partitioning parameters:* We investigate the sensitivity to the minimum bandwidth allocation, used in the bandwidth allocation algorithm presented in Section III-B2. Figure 13c shows the difference in performance with a minimum bandwidth allocation of 0GB/s, 0.5GB/s and 1GB/s, normalized to the baseline. Not using a minimum allocation leads to a 5% performance decrease compared to using a minimum allocation of 1GB/s. This is because a minimum allocation ensures a shorter queuing delay for applications that are non-memory intensive. Note that the majority of the bandwidth is still allocated to the bandwidth intensive applications. Changing the minimum allocation from 1GB/s to 0.5GB/s did not have a considerable impact on the performance of CBP.

4) *Impact of changing cache partitioning parameters:* We investigate the sensitivity to the minimum cache allocation, used in the algorithm presented in Section III-B1. Figure 13d shows the difference in performance with a minimum cache allocation of 64kB, 128kB and 256kB, normalized to the baseline. Using a minimum allocation of 64kB leads to 7.5% performance decrease compared to using a 128kB minimum allocation. One reason for the importance of the minimum allocation is that the cache hierarchy is inclusive and the L2 cache has a size of 128kB. Using a a minimum allocation of 256kB leads to a 4.2% slowdown compared to using 128kB. This is because larger minimum allocation reduces the effective capacity available for partitioning.

5) *Impact of changing prefetch sampling interval:* We finally investigate the impact of changing the *prefetch_sampling_period* used in the prefetch throttling controller III-B3. Figure 13e shows the impact of changing the sampling period on performance normalised to the baseline. The intervals we use for evaluation are 0.25ms, 0.5ms and 1ms. The advantage of using a shorter sampling period is that it carries a lower overhead, while the drawback is the risk of over/under estimating the performance benefit from prefetching. The results indicate the sampling interval of 0.5ms achieves the best performance.

VI. RELATED WORK

Isolated Management: Several techniques have been proposed in the literature that focus specifically on cache partitioning, bandwidth partitioning and prefetch throttling. Cache partitioning techniques [1]–[5] help improve performance and

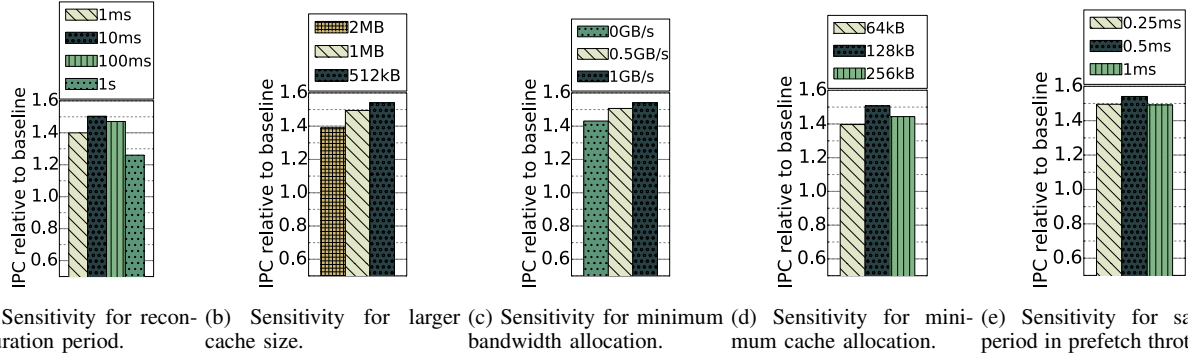


Fig. 13: Sensitivity analysis.

achieve better utilization of available cache resources, by avoiding interference among co-running applications and reducing the number of accesses to memory. Bandwidth partitioning techniques [6]–[8], [27], reduce average memory access penalty, by dynamically determining how bandwidth must be shared among the co-running applications. Prefetching can hide memory access latency by fetching the data before it is requested [9], [28]–[32]. However, inaccurate prefetches can impact application performance since it can increase the number and cost of demand misses [33]. Prefetch throttling [9], [34], involves adaptively tuning when and what prefetcher settings are used dynamically based on application characteristics and has been shown to provide better performance and address drawbacks of prefetching. The aforementioned works, consider each of the techniques in isolation and leaves room for improvement, as shown in this work, since they do not take the interaction between cache partitioning, bandwidth partitioning and prefetch throttling into account.

Coordinated Management: Several works have proposed combining two of the techniques in order to exploit the benefits from coordination. These works can be broadly classified into the following groups: i) coordinated cache and bandwidth partitioning [10], [12], ii) coordinated prefetching and cache partitioning [13], [14], and iii) coordinated bandwidth partitioning and prefetching [16]. Sahu et al. propose [10] a method for cache and bandwidth partitioning, using a CPI model for bandwidth and set partitioning for the cache. CoPart [12] combines bandwidth and cache partitioning using a user-level run-time. Unlike CBP, their goal is to improve fairness. Recently, CPpf [13] and Sun et al. [14] propose a coordinated approach for cache partitioning and prefetch where prefetch friendly applications where given a smaller cache allocation. Unlike CBP which maintains per-application partitions, cache partitioning in these two proposals is performed for groups of applications. Ebrahimi et al. [15] propose general mechanisms to make memory scheduling techniques prefetch aware. However, these works cannot use the additional interactions and trade-offs which are available when coordinately managing all three resources, which we have shown is important for performance.

Some works have also proposed coordinated management of multiple resources. For instance, CLITE [35] uses bayesian optimization to provide theoretically-grounded resource parti-

tioning to meet QoS targets of multiple resources (e.g., cores, caches, memory bandwidth, memory capacity, disk bandwidth etc.) among multiple co-located jobs. Bitirgen et al. [11] use machine learning to manage power, cache and bandwidth in a coordinated way to anticipate system-level performance impact of allocation decisions. However, in neither of these works is prefetch throttling considered, which we have shown is important in order to realise the full potential of coordinated resource management. To the best of our knowledge, CBP is the first coordinated resource manager for cache partitioning, bandwidth partitioning and prefetch throttling.

VII. CONCLUSIONS

We have presented CBP, a mechanism for coordinated management of cache partitioning, bandwidth partitioning and prefetch throttling. The design is motivated by our in-depth characterisation of the performance impact of cache, bandwidth and prefetch allocation and their interactions. CBP combines local resource allocation controllers with a coordination mechanism that dynamically manages and allocates the resources, in a way which considers both inter- and intra-application interactions. Our evaluation on a tiled 16-core CMP demonstrates that CBP improves performance by up to 86% (geo. mean 50%) compared to a system without partitioning and prefetching and by up to 36% (geo. mean 11%) over the state-of-the-art technique that manages cache partitioning and prefetching in a coordinated manner.

VIII. ACKNOWLEDGEMENTS

This research has been funded by the Swedish Research Council (VR) under the projects ACE (registration number 621-2014-6221) and PRIME (registration number 2019-04929). In addition, the work has received funding from the European Union Horizon 2020 research and innovation programme under the LEGaTO project with grant agreement No. 780681, and the MECCA project with contract number ERC2013-AdG 340328. The simulations were enabled by resources provided by the Swedish National Infrastructure for Computing (SNIC) at the Chalmers Centre for Computational Science and Engineering (C3SE) partially funded by the Swedish Research Council through grant agreement no. 2018-05973.

REFERENCES

- [1] M. K. Qureshi and Y. N. Patt, "Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches," in *Proc. MICRO-49*, 2006.
- [2] H. Lee, S. Cho, and B. R. Childers, "CloudCache: Expanding and shrinking private caches," in *Proc. HPCA-17*, 2011.
- [3] N. El-Sayed, A. Mukkara, P. A. Tsai, H. Kasture, X. Ma, and D. Sanchez, "Kpart: A hybrid cache partitioning-sharing technique for commodity multicores," vol. 2018-Febru, 2018, pp. 104–117.
- [4] R. Manikantan, K. Rajan, and R. Govindarajan, "Probabilistic shared cache management (PriSM)," in *Proc. ISCA-39*, 2012.
- [5] D. Sanchez and C. Kozyrakis, "Vantage: Scalable and Efficient Fine-Grain Cache Partitioning," *Proc. ISCA-38*, 2011.
- [6] J. Park, S. Park, M. Han, J. Hyun, and W. Baek, "Hypart: A hybrid technique for practical memory bandwidth partitioning on commodity servers." Institute of Electrical and Electronics Engineers Inc., 11 2018, pp. 1–14.
- [7] D. R. Hower, H. W. Cain, and C. A. Waldspurger, "Pabst: Proportionally allocated bandwidth at the source and target," in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 505–516.
- [8] F. Liu, X. Jiang, and Y. Solihin, "Understanding how off-chip memory bandwidth partitioning in chip multiprocessors affects system performance," in *HPCA - 16 2010 The Sixteenth International Symposium on High-Performance Computer Architecture*, 2010, pp. 1–12.
- [9] F. Dahlgren, M. Dubois, and P. Stenstrom, "Fixed and adaptive sequential prefetching in shared memory multiprocessors," in *1993 International Conference on Parallel Processing - ICPP'93*, vol. 1, 1993, pp. 56–63.
- [10] A. Sahu and S. Ramakrishna, "Creating heterogeneity at run time by dynamic cache and bandwidth partitioning schemes." Association for Computing Machinery, 2014, pp. 872–879.
- [11] R. Bitirgen, E. Ipek, and J. F. Martínez, "Coordinated management of multiple interacting resources in chip multiprocessors: A machine learning approach," 2008, pp. 318–329.
- [12] J. Park, S. Park, and W. Baek, "Copart: Coordinated partitioning of last-level cache and memory bandwidth for fairness-aware workload consolidation on commodity servers," vol. 19. ACM.
- [13] J. Xiao, A. D. Pimentel, and X. Liu, "Cppf: A prefetch aware llc partitioning approach." Association for Computing Machinery, 8 2019, pp. 1–10.
- [14] G. Sun, J. Shen, and A. V. Veidenbaum, "Combining prefetch control and cache partitioning to improve multicore performance," *Proceedings - 2019 IEEE 33rd International Parallel and Distributed Processing Symposium, IPDPS 2019*, pp. 953–962, 5 2019.
- [15] E. Ebrahimi, C. J. Lee, O. Mutlu, and Y. N. Patt, "Prefetch-aware shared resource management for multi-core systems," *ACM SIGARCH Computer Architecture News*, vol. 39, p. 141, 2011.
- [16] F. Liu and Y. Solihin, "Studying the impact of hardware prefetching and bandwidth partitioning in chip-multiprocessors." Association for Computing Machinery (ACM), 2011, p. 37.
- [17] T. E. Carlson, W. Heirman, S. Eyerman, I. Hur, and L. Eeckhout, "An evaluation of high-level mechanistic core models," *ACM TACO*, 2014.
- [18] K. J. Nesbit and J. E. Smith, "Data cache prefetching using a global history buffer," *IEEE Micro*, vol. 25, no. 1, pp. 90–97, 2005.
- [19] Y. Ishii, M. Inaba, and K. Hiraki, "Access map pattern matching for data cache prefetch," in *Proceedings of the 23rd International Conference on Supercomputing*, ser. ICS '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 499–500.
- [20] "Intel® 64 and ia-32 architectures developer's manual: Vol. 3b." [Online]. Available: <https://www.intel.com/content/www/us/en/architecture-and-technology/64-ia-32-architectures-software-developer-vol-3b-part-2-manual.html>
- [21] N. Holtryd, M. Manivannan, P. Stenström, and M. Pericàs, "Delta: Distributed locality-aware cache partitioning for tile-based chip multiprocessors," in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2020, pp. 578–589.
- [22] "Intel® 64 and IA-32 Architectures Software Developer Manuals." [Online]. Available: <https://software.intel.com/content/www/us/en/develop/articles/intel-sdm.html#nine-volume>
- [23] "Introduction to Memory Bandwidth Allocation." [Online]. Available: <https://software.intel.com/content/www/us/en/develop/articles/introduction-to-memory-bandwidth-allocation.html>
- [24] N. Muralimanohar, R. Balasubramonian, and N. Jouppi, "Optimizing nuca organizations and wiring alternatives for large caches with cacti 6.0," in *Proc. MICRO-40*, 2007.
- [25] T. Sherwood, E. Perelman, G. Hamerly, and B. Calder, "Automatically characterizing large scale program behavior," in *Proc. ASPLOS-10*, 2002.
- [26] N. Beckmann and D. Sanchez, "Jigsaw: Scalable software-defined caches," in *Proc. PACT-22*, 2013.
- [27] Y. Xiang, C. Ye, X. Wang, Y. Luo, and Z. Wang, "Emba: Efficient memory bandwidth allocation to improve performance on intel commodity processor," in *Proceedings of the 48th International Conference on Parallel Processing*, ser. ICPP 2019. New York, NY, USA: Association for Computing Machinery, 2019.
- [28] F. Dahlgren and P. Stenström, "Evaluation of hardware-based stride and sequential prefetching in shared-memory multiprocessors," vol. 7. IEEE Comput. Soc. Press, 1996, pp. 385–398.
- [29] K. Nesbit, A. Dhodapkar, and J. Smith, "Ac/dc: an adaptive data cache prefetcher." IEEE, 2004, pp. 135–145.
- [30] K. J. Nesbit and J. E. Smith, "Data cache prefetching using a global history buffer," *IEEE Micro*, vol. 25, pp. 90–97, 1 2005.
- [31] J. Lee, H. Kim, and R. Vuduc, "When prefetching works, when it doesn't, and why," *ACM Transactions on Architecture and Code Optimization*, vol. 9, pp. 1–29, 3 2012.
- [32] S. Kondguli and M. Huang, "Division of labor: A more effective approach to prefetching." IEEE, 6 2018, pp. 83–95.
- [33] E. Ebrahimi, O. Mutlu, C. J. Lee, and Y. N. Patt, "Coordinated control of multiple prefetchers in multi-core systems," 2009, p. 316.
- [34] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, "Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers," in *Proceedings of the 2007 IEEE 13th International Symposium on High Performance Computer Architecture*, ser. HPCA '07. USA: IEEE Computer Society, 2007, p. 63–74.
- [35] T. Patel and D. Tiwari, "Clite: Efficient and qos-aware co-location of multiple latency-critical jobs for warehouse scale computers," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 193–206.